

The projects are complementary, not redundant. Churilov is much larger and stronger on security impact; our study is substantially stronger on measurement validity, replication fidelity, and failure-mode analysis.

I would still proceed with our paper—but position it as a methodological correction and deeper replication, not simply another frontier-model update.

Side-by-side

Dimension	Churilov et al.	Our Campaign 4
Scale	199,845 generations; ~39,969 per model	800 prompts per model
Languages	Python and JavaScript	Python
Models	Five hosted frontier models	Hosted frontier plus local/open models
Recommendation extraction	Install commands and imports found in generated code	Original three-channel design in Track A; explicit two-query recommendation protocol in Track B
Registry validation	Original master lists, checked April 2026	Frozen 2024 and August 2026 registries
Parser validation	Heuristic regexes and noise filter	Frozen grammar, manually validated sample, malformed-response accounting
Uncertainty	Package-mention Wilson intervals and mention-level tests	Prompt-cluster bootstrap and paired prompt analysis
Security contribution	Universal names, registrability testing, coordinated disclosure	Concentration, malformed-output sensitivity, context-cap diagnostics
Main strength	Breadth and operational attack surface	Measurement and causal interpretation

The paper is available locally here: `:codex-file-citation{path="C:/MachineLearning/github/PackageHallucination/churilov-2605.17062v3.pdf" purpose="source"}` The authors also provide an [arXiv version](#) and a [public artifact repository](#).

The central problem with their “faithful replication”

Their implementation generates code and extracts package names from installation commands and imports. It does not reproduce the original experiment’s two additional package-query channels:

- 1. Packages required by the generated code.
- 2. Packages useful for solving the original prompt.

Those channels are part of the original headline measurement described in our [replication documentation](#). Consequently, Churilov’s 2026 percentages are not directly comparable to the original percentages—or to our Track B rates.

Their experiment is better described as a very large code-import hallucination study using the original prompt corpus.

That remains valuable, but it changes what the headline measures.

The “range compression” claim is overstated

Churilov reports a narrow overall range of 4.62%–6.10%, or 5.49%–7.27% when restricted to Python.

There are three qualifications:

- Their headline comparison uses the original study’s 5.2% commercial and 21.7% open-source category averages as though those were the original model-level endpoints. The actual original model range was approximately 3.59%–26.12%.
- They selected five relatively similar hosted frontier systems, whereas the original study deliberately covered a heterogeneous 16-model cohort.
- Their extraction protocol differs from the original protocol.

So “the selected 2026 frontier cohort produced a narrow code-extraction range” is supported. “The model-to-model hallucination range has compressed eleven-fold” is not a defensible like-for-like conclusion.

Our modern-model range—1.31% to 17.91%—also demonstrates that a broader frontier/local cohort does not necessarily exhibit compression. But we should not present our range as a direct contradiction because our task and denominator differ.

Their inferential statistics need reanalysis

The artifact appears to treat individual package mentions as independent observations for Wilson intervals, chi-square tests, and permutation tests. Package mentions are nested inside responses and prompts; one response can contribute multiple correlated observations.

That is pseudoreplication. A permutation test does not fix it if the shuffled unit remains the package mention.

Their model differences are often small—roughly tenths of a percentage point—so the five reported significant pairwise comparisons should be rerun using the prompt as the resampling or permutation unit. They may survive, but the published intervals and p-values cannot establish that as currently computed.

Our prompt-cluster bootstrap is a meaningful methodological advantage.

What their work genuinely adds

The most important result is not their rate comparison. It is the cross-model security surface:

- 127 nonexistent names appeared across all five models.
- 53 were reportedly registrable after coordinated disclosure.
- They tested Python and JavaScript.
- Their full-corpus scale is roughly 50 times ours per model.
- They undertook coordinated disclosure rather than merely reporting candidate names.

That is an excellent operational contribution and should be credited clearly in our related-work section.

Their overlap interpretation is less secure. Shared names can arise from shared training data, but all models also received the same prompts. Some prompts directly imply package-family names—for example, their AWS CDK example—so “universal” hallucinations may partly be prompt-induced rather than evidence of shared training contamination.

Our preliminary cross-model analysis reinforces this concern: Campaign 4 has five names shared across all six models, but four of those five arise across every model on the same prompt. We should not disclose the names, but the pattern strongly motivates prompt-conditioned overlap analysis.

What I recommend adding to our paper

1. A code-only bridge analysis.

Re-extract install/import package names from our existing generated code using a Churilov-compatible heuristic. No new model calls are needed. Report those results separately with prompt-cluster confidence intervals. This would provide the closest available comparison.

2. Prompt-conditioned cross-model overlap.

Report both ordinary Jaccard overlap and overlap conditioned on the same prompt. Separate:

- same name, same prompt;
- same name, different prompts;
- universal names whose occurrence is dominated by one prompt.

This directly improves on Churilov’s training-data interpretation.

3. A measurement crosswalk.

Give each rate an explicit numerator, denominator, extraction channel, registry date, language, and malformed-output policy. This will prevent readers from casually comparing incompatible percentages.

4. A clustered reanalysis recommendation.

State—without overclaiming—that Churilov’s model comparisons require prompt-level inference before their significance claims can be interpreted confidently.

5. A complementary framing.

Present the three studies as answering different questions:

- Original work: How often do heterogeneous models recommend nonexistent packages?
- Churilov: What code-import hallucination surface exists across a large 2026 hosted-model cohort?
- Our work: Which findings replicate under validated measurement, and how do model family, output validity, concentration, and inference choices affect the result?

Bottom line

We compare well. They have the stronger scale and security-disclosure story; we have the stronger experimental-control and measurement-validity story.

Their publication raises the value of ours because it creates a timely methodological issue: the field is now reporting superficially similar “package hallucination rates” produced by materially different extraction channels and statistical units. Our paper can clarify that confusion while contributing a broader model cohort, validated parsing, dual-registry checks, clustered inference, and prompt-conditioned overlap.

I recommend doing the code-only bridge and prompt-conditioned overlap analyses before drafting the full paper. Those two additions would turn the relationship from “another replication” into a direct and constructive response to the new literature.